Prototipo 1 — espivota

Equipo

Dato	Información
Equipo	Fabian Espitia y asociados
Nombre	2c

Integrantes

Nombre completo	Correo
Fabián Espitia Sotelo	fespitia@unal.edu.co
Juan Sebastian Moya Alvarez	umoyaa@unal.edu.co
Sergio Nicolas Rey Mora	sreym@unal.edu.co
Kevin David Rodriguez Riveros	kerodriguezr@unal.edu.co
José Julián Pira Naranjo	Jpira@unal.edu.co
Juan Camilo Camargo Prieto	jucamargop@unal.edu.co
Andrés Felipe Rosada Mosquera	arosada@unal.edu.co
Cristhian Yamith Cely Oliveros	ccelyo@unal.edu.co

Sistema de software

espivota

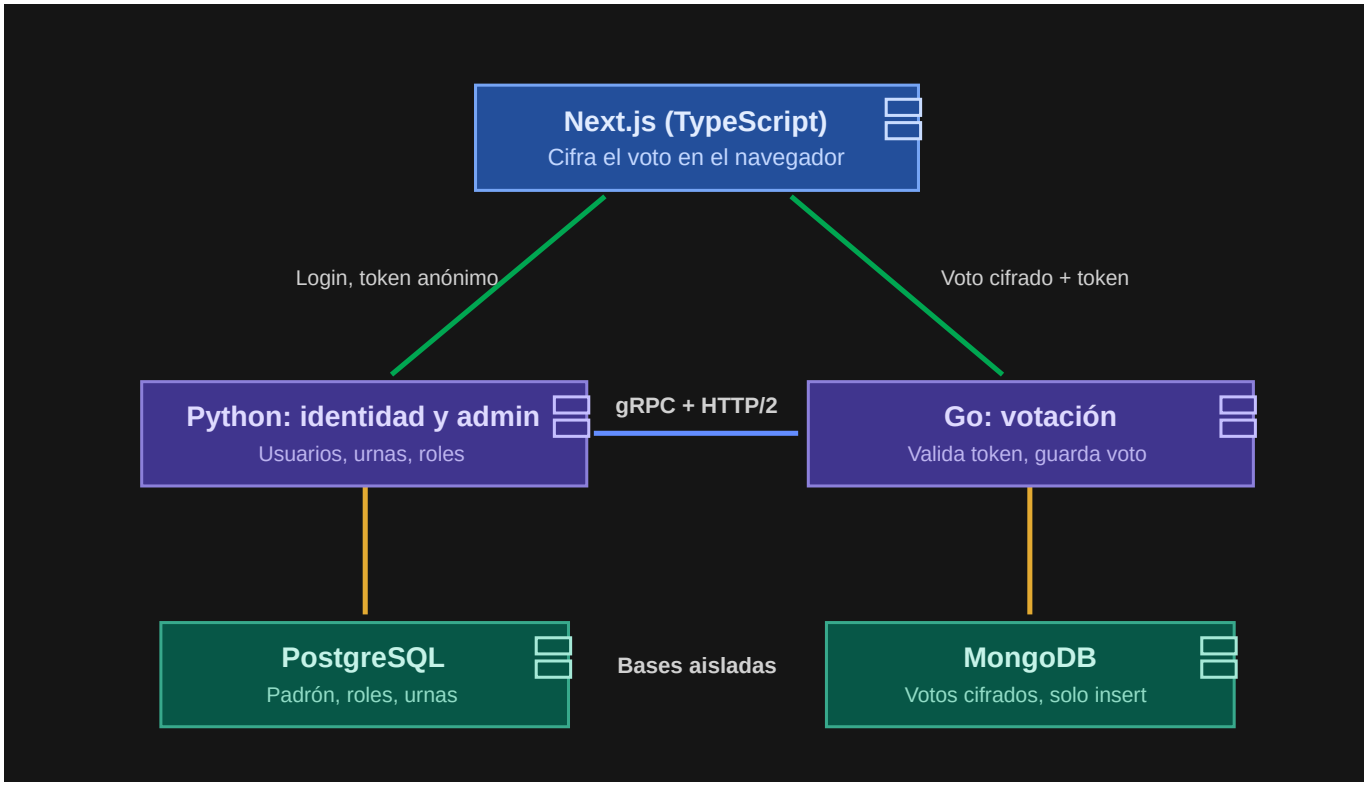


Repositorio: github.com/ccelyo/Votaciones
Prototipo desplegado: espivota.vercel.app

espivota es un sistema web distribuido para administrar elecciones y recibir votos electrónicos. Los administradores gestionan urnas, opciones y el padrón; los votantes se autentican, consultan las elecciones disponibles y envían una papeleta cifrada desde el navegador. El sistema separa la autenticación de la recepción del voto: Identity conoce al votante, mientras que Ballot recibe el token anónimo y la papeleta cifrada, y devuelve un recibo verificable. El prototipo contempla autenticación con MFA, emisión de tokens mediante firma ciega RSA (RFC 9474) y recibos firmados con Ed25519.

Estado y alcance: la arquitectura define el envío de una papeleta cifrada y el cegado del token en el navegador.

Estructura de componentes y conectores (C&C)



Estilos arquitectónicos

- **Cliente-servidor:** el navegador consume las API remotas de Identity y Ballot mediante HTTP.
- **Microservicios:** `identity` y `ballot` son servicios de lógica independientes, implementados en Python y Go, respectivamente, y con responsabilidades diferenciadas.
- **Base de datos por servicio:** PostgreSQL es propiedad de `identity` y MongoDB de `ballot`; ningún servicio consulta directamente la base del otro.
- **Petición-respuesta y RPC:** la interfaz se comunica con los servicios por REST/JSON sobre HTTP; la comunicación administrativa entre servicios usa gRPC sobre HTTP.
- **Separación entre identidad y voto:** el navegador ciega el mensaje del token y lo desciega localmente. Identity firma el mensaje cegado sin conocer el token final; Ballot valida el token anónimo y almacena la papeleta cifrada, sin recibir las credenciales ni la sesión del votante. La API pública de Ballot se consume sin cookies.

- **Despliegue orientado a contenedores:** los servicios y las bases de datos se ejecutan en contenedores Docker.

Elementos

Elemento	Tipo	Tecnología	Responsabilidad
`platform`	Presentación	Next.js y TypeScript	Interfaz de votantes y administradores; consulta urnas y contiene el módulo criptográfico previsto para cifrar la papeleta y cegar/descifrar el token. En esta iteración, el cifrado está simulado y RSABSSA aún requiere completar la interoperabilidad.
`identity`	Lógica	Python y FastAPI	Login y MFA, sesiones y roles, urnas, padrón electoral y firma ciega del token. Administra PostgreSQL y llama a Ballot por gRPC.
`ballot`	Lógica	Go	API anónima de votación; valida y consume el token, persiste la papeleta recibida y firma el recibo Ed25519. La verificación de la prueba de papeleta es estructural en esta iteración. También ofrece la API gRPC administrativa.
`postgres`	Datos relacionales	PostgreSQL 16	Usuarios, roles, elecciones, padrón y registro de emisión de tokens de Identity.
`mongo`	Datos NoSQL	MongoDB 8	Parámetros de elecciones publicados y papeletas cifradas de Ballot; permite evitar el doble uso de un token.

Conectores

Conector	Origen → destino	Protocolo	Contenido
C1	`platform` → `identity`	REST/JSON sobre HTTP/1.1	Credenciales y MFA; sesión bearer; consulta de urnas y parámetros; solicitud de firma del mensaje cegado.
C2	`platform` → `ballot`	REST/JSON anónimo sobre HTTP/1.1, sin cookies	Consulta de parámetros públicos y envío de token anónimo con papeleta cifrada;

Conector	Origen → destino	Protocolo	Contenido
			respuesta con recibo.
C3	`identity → ballot`	gRPC sobre HTTP/2	Registro y cierre de urnas, consulta de estado y conteo de papeletas. En este prototipo la conexión gRPC interna no usa mTLS.
C4	`identity → postgres`	SQL mediante psycopg	Lectura y escritura de usuarios, roles, elecciones, padrón y emisión de tokens.
C5	`ballot → mongo`	Protocolo MongoDB mediante driver	Lectura de parámetros de urna y persistencia de papeletas cifradas y datos de control del token.

Instrucciones de ejecución

Prototipo desplegado

Abrir https://espivota.vercel.app/. Para las pruebas del equipo se proporcionan los usuarios `admin1`–`admin4` y `votante1`–`votante10`; la contraseña es `prueba123`.

Ejecución local de la interfaz

Requisitos: Git, Node.js y npm.

```
git clone https://github.com/ccelyo/Votaciones.git
cd Votaciones/platform
cp env.example .env.local
npm ci
npm run dev
```

La interfaz queda disponible en http://localhost:3000. La configuración incluida en `env.example` usa el modo *mock*, adecuado para explorar la interfaz en un navegador, pero no envía votos a los servicios reales.

Contenedores de servicios

Requisitos: Git, Docker y Docker Compose. Desde la raíz del repositorio, levanta el entorno local completo —interfaz, servicios de identidad y votación, y sus bases de datos— con:

```
git clone https://github.com/ccelyo/Votaciones.git
cd Votaciones
docker compose up --build
```

Compose inicializa PostgreSQL con el esquema y los usuarios de prueba, inicia MongoDB como *replica set* para soportar transacciones, y configura la interfaz para usar las API reales.

Servicio	URL local	Función
`platform`	http://localhost:3000	Interfaz web
`identity`	http://localhost:8000	API de identidad y administración
`ballot`	http://localhost:8080	API anónima de votación
`postgres`	Solo red interna de Docker	Base relacional de Identity
`mongo`	Solo red interna de Docker	Base NoSQL de Ballot

El puerto gRPC `9091` de Ballot solo se usa dentro de la red de Docker; no se publica en el host. Los usuarios de prueba son `admin1`–`admin4` y `votante1`–`votante10`, con contraseña `prueba123` y código MFA `123456`. Para cambiar el puerto web si `3000` ya está ocupado:

```
WEB_PORT=3100 docker compose up --build
```

Detén los servicios sin borrar sus datos con `docker compose down`. Para eliminar también los volúmenes de bases de datos y claves y empezar de nuevo:

```
docker compose down -v
```

Estas credenciales y la clave de recibos predeterminada son únicamente para desarrollo local; no deben usarse en un despliegue público.